

Business 41204

Machine Learning

Introduction to Deep Learning

Outline:

1. Introduction
2. Neural Networks
3. House Price Example Revisited
4. Transfer Learning with X-Rays

1. Introduction

Biological Inspiration

(Artificial) neural networks inspired by biological networks

Donald Hebb (1949) *The Organization of Behavior*

- How do humans and animals match patterns with missing information?

“When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A’s efficiency, as one of the cells firing B, is increased.”

Inspired early work on artificial networks such as the perceptron

- Rosenblatt (1958), “The Perceptron: A Probabilistic Model for Information Storage and Organization of the Brain,” *Psychological Review*, 65(6): 386-408.

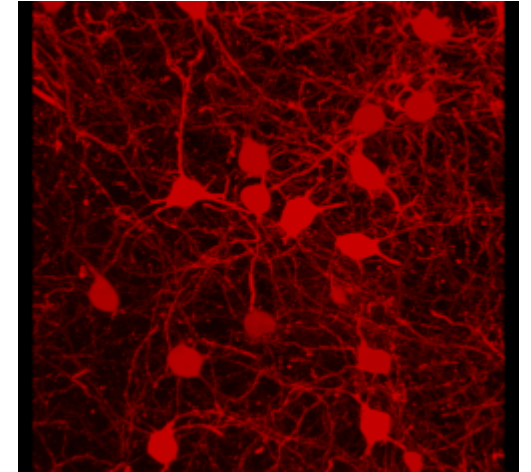
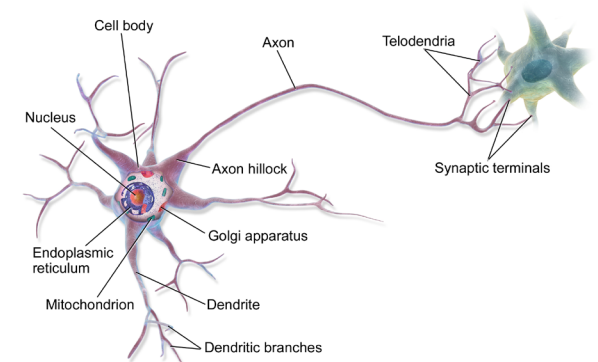


Image of (part of) mouse neural network
https://en.wikipedia.org/wiki/File:Projections_of_Gpr101_TomatoMSNsInSTR.gif



Neuron

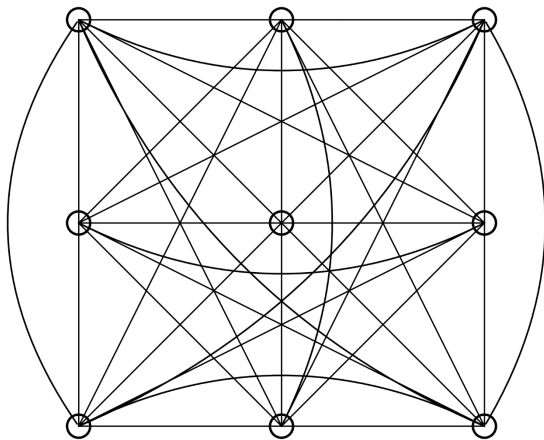
Hopfield Network (1982)

Simple network in which every neuron connects to every other neuron

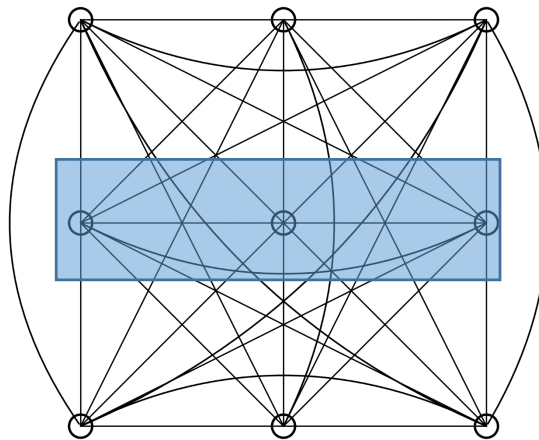
- Imagine each neuron represents a pixel in an image

Training

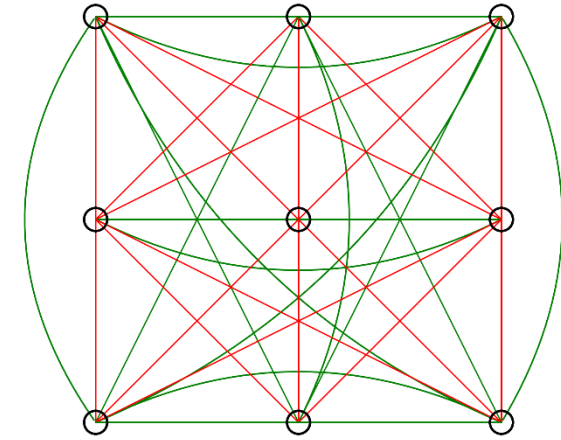
- Neurons fire if they detect light in their pixel
- When neurons fire together, they build a stronger connection
 - Start with $connection(i,j) = 0$
 - Update to $connection(i,j) = connection(i,j) + \begin{cases} 1 & i,j \text{ both fire} \\ -1 & \text{otherwise} \end{cases}$



Original Network



Image



Updated Network

Hopfield Network (1982)

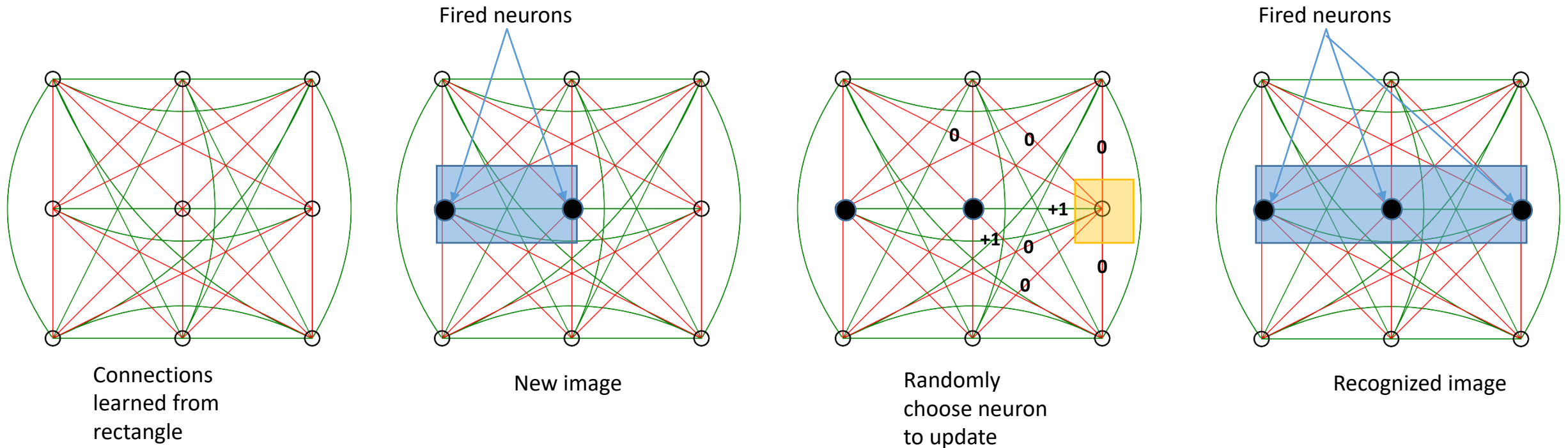
Recall – given an input, recall “correct memory”

- Present machine with new image
- Neurons fire if they detect light in their pixel
 - Write $F_i = 1$ if neuron i fires
- Other neurons update depending on learned connections
 - Neuron i updates using $z_i = \sum_j \text{connection}(i, j)F_j$
 - After update, i fires if $z_i > 0$ (and does not fire otherwise)
- Eventually settle on a learned image (hopefully)

Machine learns to recognize images without explicit instructions!

Hopfield Network (1982)

Illustration of recall:



Numbers show $F * connection$

Machine learns to recognize images without explicit instructions!

Less Trivial Example

Hopfield Net trained to remember A and B

Learned to recognize A and B without needing “formula” for “A”-ness and “B”-ness

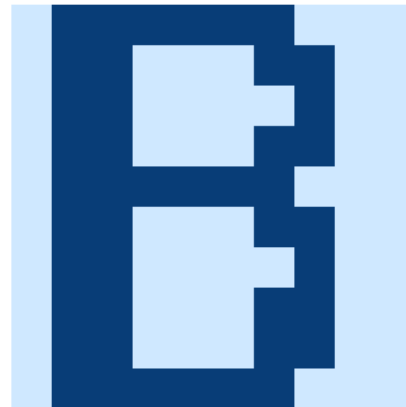
Initial A



Initial corrupted A



Initial B



Initial corrupted B



2. Deep Neural Networks

Single Layer Neural Network as a Function

Modern neural networks only loosely tied to biological motivation

- Functions for taking p -dimensional input X to produce target output

Simple single layer neural network:

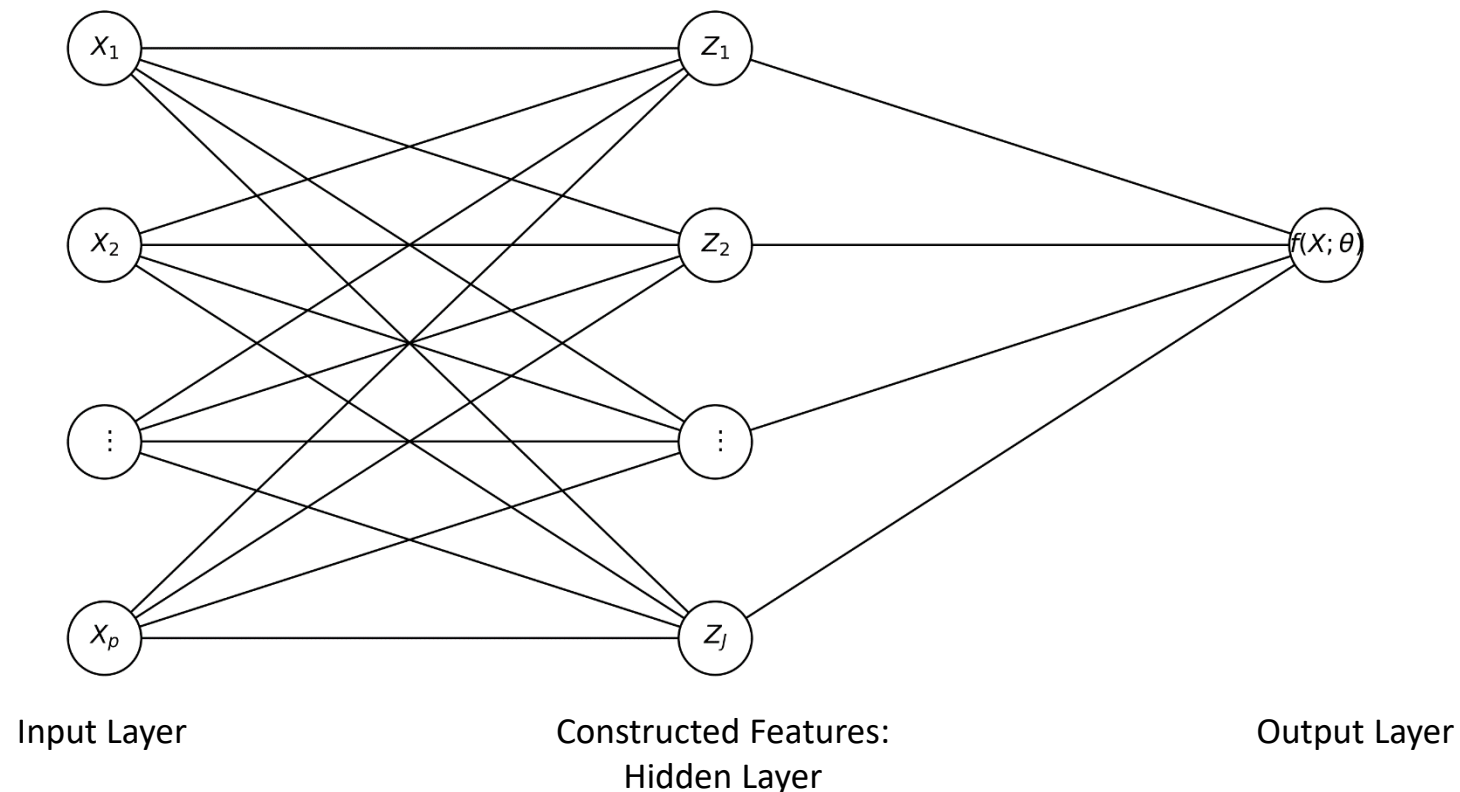
$$f(X, \theta) = b^{(2)} + \sum_{j=1}^J w_j^{(2)} \hat{Z}_j \text{ where } \hat{Z}_j = \sigma_j \left(b_j^{(1)} + \sum_{k=1}^p w_{j,k}^{(1)} X_k \right)$$

- θ captures all the **parameters** of the neural network
 - “**biases**”: $b_1^{(1)}, \dots, b_J^{(1)}, b^{(2)}$
 - “**weights**”: $w_{1,1}^{(1)}, \dots, w_{1,p}^{(1)}, \dots, w_{J,1}^{(1)}, \dots, w_{J,p}^{(1)}, w_1^{(2)}, \dots, w_J^{(2)}$
- $\hat{Z}_j$ are J constructed features (“activations” obtained from “neurons”)
 - “**activation functions**”: $\sigma_j(\cdot)$. (Typically chosen to be the same, but don’t have to be)
 - To be “interesting,” need to be nonlinear (**Why?**)
 - **Feature engineering**: rather than rely on raw features (or known transformations), learn them from the data
- Output in this simple example is a linear combination of the constructed features

Single Layer Neural Network Schematic

Rather than write the mathematical expression, neural networks are often represented schematically

Here's the neural network from the previous slide



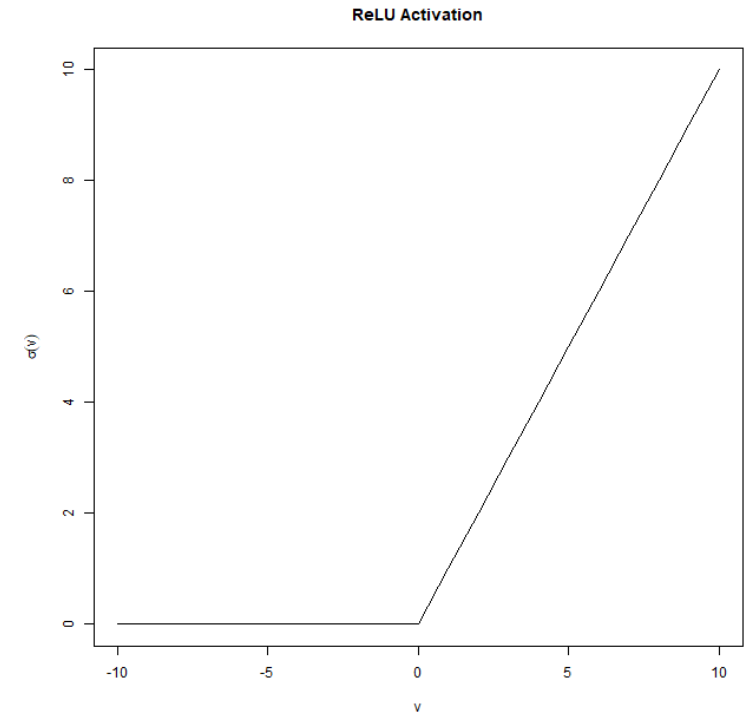
Activation Functions

Most popular activation function (currently) is the **rectified linear unit** (ReLU):

$$\sigma(v) = \max(0, v)$$

Other common activation functions:

- Smoothed ReLU (SReLU): $\sigma(v) = \log(1 + \exp(v))$
- Sigmoid: $\sigma(v) = \frac{1}{1 + \exp(-v)}$



Deep Neural Networks

Deep Neural Networks (DNN; aka **Deep Nets**, **Deep Learning**) add more **layers** to the NN structure

- DNN structure is repeated composition of nonlinear mappings

$$X \xrightarrow{f_1} H^{(1)} \xrightarrow{f_2} \dots \xrightarrow{f_m} H^{(m)} \xrightarrow{f_{m+1}} Z$$

- **Neurons:** $H^{(l)} = \{H_k^{(l)}\}_{k=1}^{K_l}$
- Mapping from one layer to the next:

$$f_l: v \rightarrow \{H_k^{(l)}\}_{k=1}^{K_l} = \left\{ \sigma_k^{(l)} \left(b_k^{(l)} + v' w_k^{(l)} \right) \right\}_{k=1}^{K_l}$$

- Input layer: X (raw predictor variables)
- Hidden layers: Layers between input and output
- Final model uses Z as inputs (e.g. $\hat{f}(X) = b_0 + Z' w_0$)
- θ collects all the parameters (weights, biases, and parameters of final model)

DNN Schematics

Schematics for some deep neural network architectures

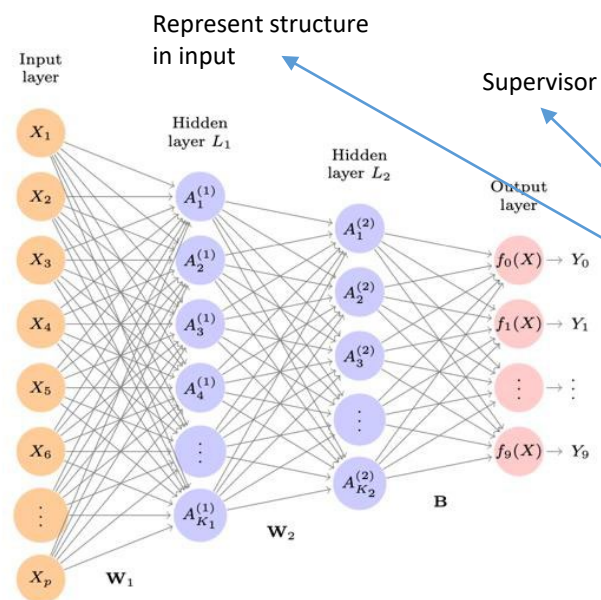


FIGURE 10.4. Neural network diagram with two hidden layers and multiple outputs, suitable for the MNIST handwritten-digit problem. The input layer has $p = 784$ units, the two hidden layers $K_1 = 256$ and $K_2 = 128$ units respectively, and the output layer 10 units. Along with intercepts (referred to as biases in the deep-learning community) this network has 235,146 parameters (referred to as weights).

Reproduced from James, Witten, Hastie, Tibshirani, and Taylor (2023), *An Introduction to Statistical Learning with Applications in Python*

DNNs are a “combination” of unsupervised and supervised learning.

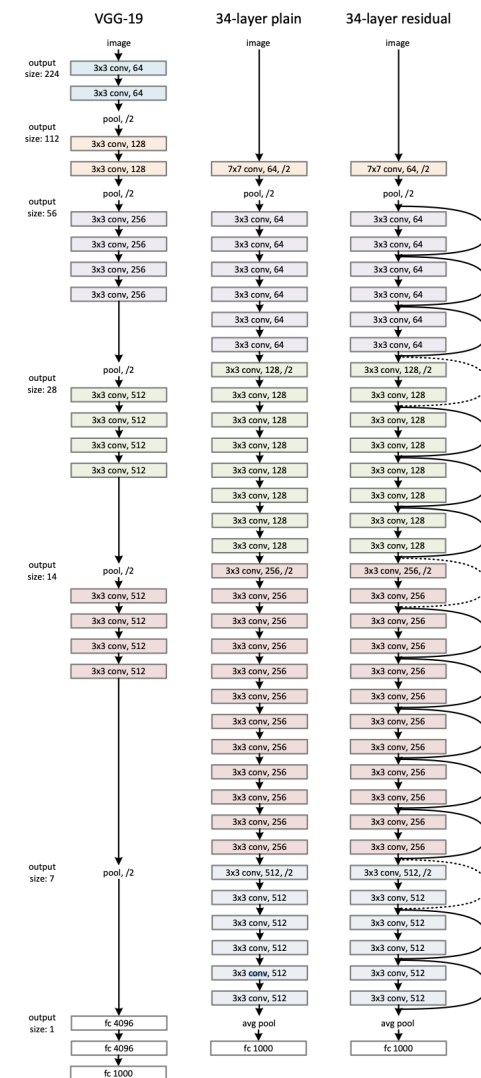
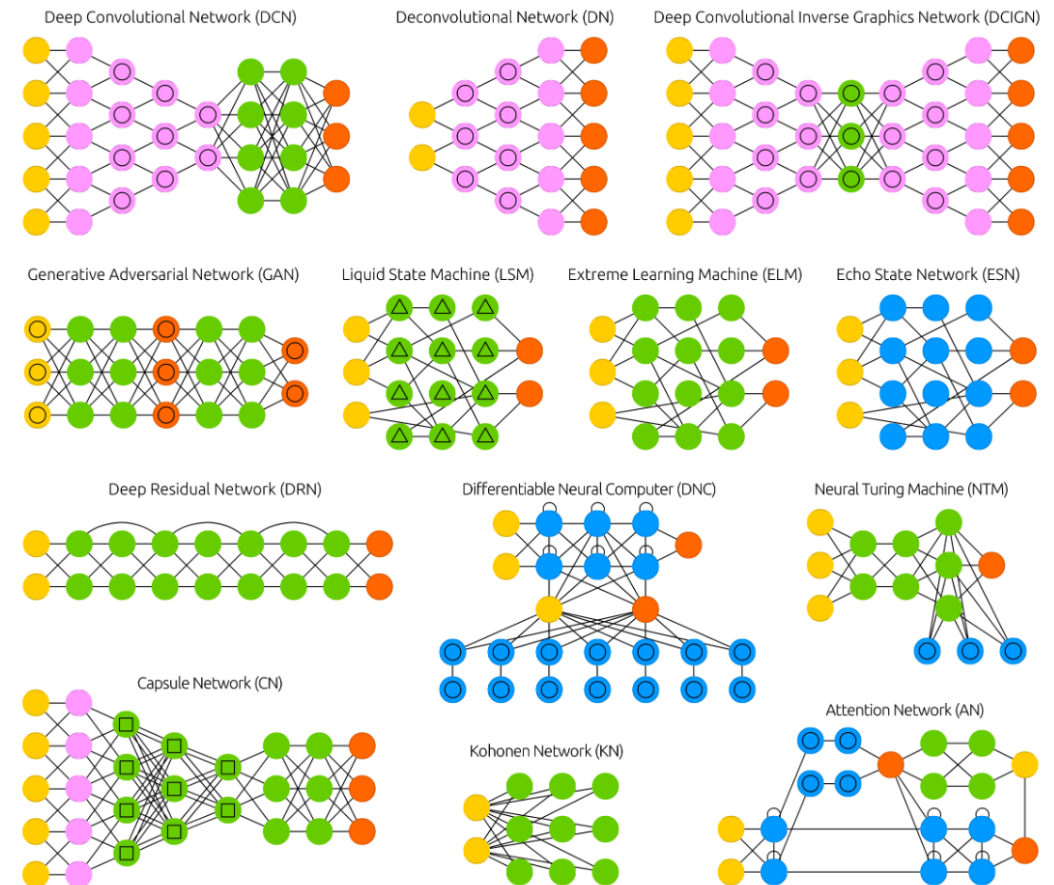
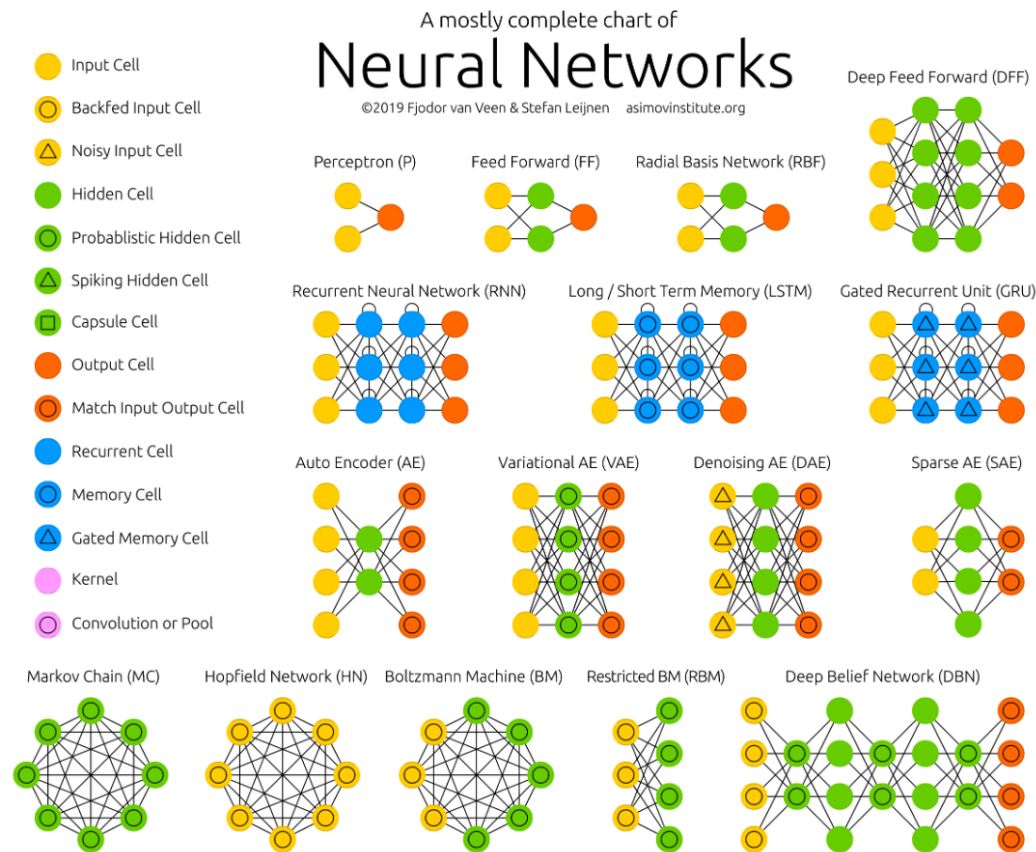


Figure 3. Example network architectures for ImageNet. **Left:** the VGG-19 model [41] (19.6 billion FLOPs) as a reference. **Middle:** a plain network with 34 parameter layers (3.6 billion FLOPs). **Right:** a residual network with 34 parameter layers (3.6 billion FLOPs). The dotted shortcuts increase dimensions. **Table 1** shows more details and other variants.

Reproduced from He, Zhang, Ren, and Sun (2015) “Deep Residual Learning for Image Recognition” (<https://arxiv.org/pdf/1512.03385.pdf>)

More DNNs

Many DNN structures designed with different tasks in mind



DNN Problem

Train DNN by solving

$$\min_{\theta} \sum_i \text{loss}(Y_i, \hat{f}(X_i, \theta)) + \text{penalty}(\lambda, \theta)$$

- $\hat{f}(X, \theta)$ is entire network structure with all parameters
- loss is a sensible loss function for target (e.g. squared errors)
- penalty is a penalty function over model parameters (e.g. sum of absolute values, ℓ_1 , or squares, ℓ_2) with penalty parameter λ
- generally non-convex (hard to optimize)
- lots of tuning choices
 - Depth of network
 - Neurons per layer
 - Connections between neurons
 - Optimization parameters (learning rate, stopping rule, ...)
 - Penalty parameters
 - ...
 - Means in practice use combination of received wisdom, trial and error, and lots of computation

Optimization: Gradient Descent

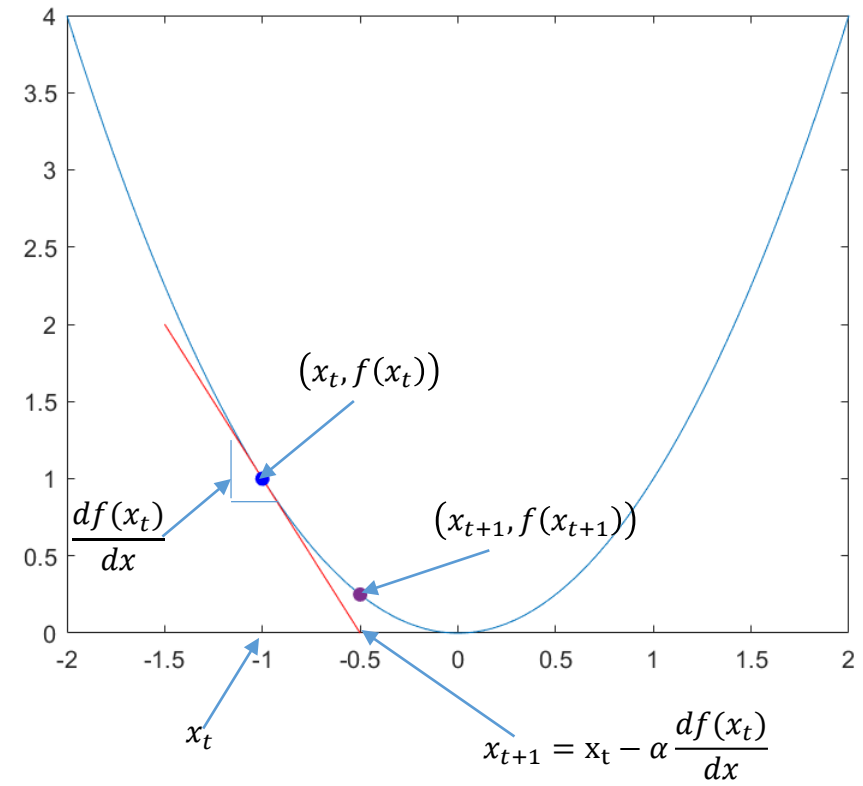
Gradient descent is a simple numerical method for optimizing a function.

Suppose we wish to minimize function $f(x)$ with derivative $\frac{df(x)}{dx}$

- From candidate value x_t , we can update to a new value, x_{t+1} , (hopefully closer to minimum) by following derivative

$$x_{t+1} = x_t - \alpha \frac{df(x_t)}{dx}$$

- α is the **learning rate** (or **step size**)
- From any **starting point**, we repeat until some **stopping criterion** is met
- Easily generalized to functions of more than one variable



The exact numbers in the figure are $x_t = -1$, $\frac{df(x_t)}{dx} = -2$, $\alpha = .25$, $x_{t+1} = -.5$

Stochastic Gradient Descent (SGD)

For many problems (including DNNs), calculating the gradient is “costly”

- Clever methods for calculating gradients for NNs ([back propagation](#))
- BUT, involves summing gradient over all observations

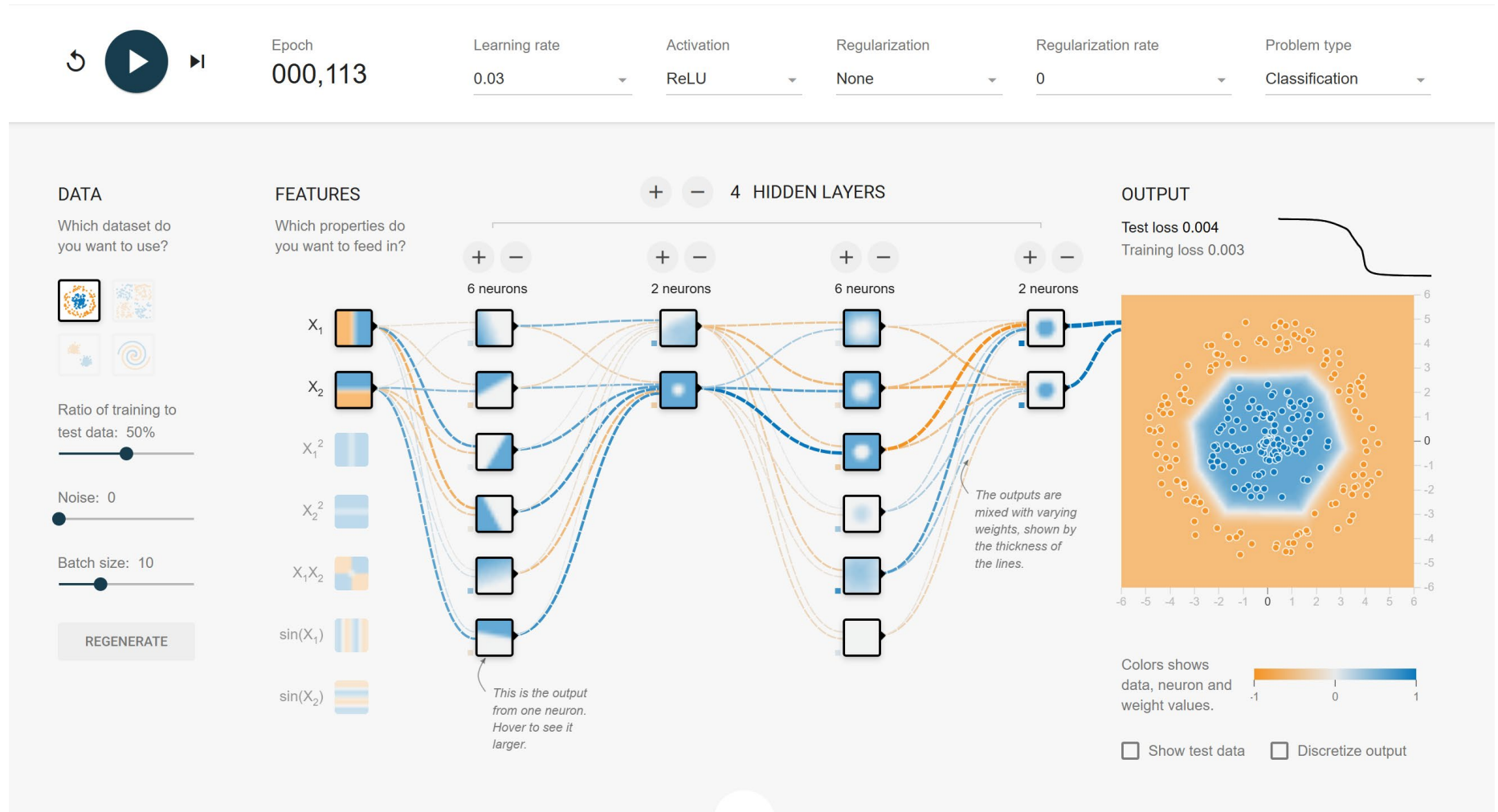
SGD uses subsamples ([batches](#)) of data to compute gradient

- Each pass through the data called an [epoch](#)
- Aids in computation: Don't need to have all data in memory
- Adds levers for regularization
 - [Dropout](#): drop neuron/treat as zero during gradient calculation
 - Forces gradient to be simpler, helps avoid neurons over-specializing to training instances
- Noise in gradients may have some advantages for optimizing complex functions

Visualizing DNNs

Probably easier to
just visualize what is
going on

playground.tensorflow.org



3. House Price Prediction Revisited

Training a Simple DNN from Scratch

House Price Example (Again)

Goal: Illustrate training DNN models based on Seattle house price data

Features:

- `sqft_living, sqft_lot, sqft_above, sqft_basement, bathrooms, bedrooms, floors, waterfront, view, condition, yr_built, yr_renovated, city (44 categories), statezip (77 categories)`

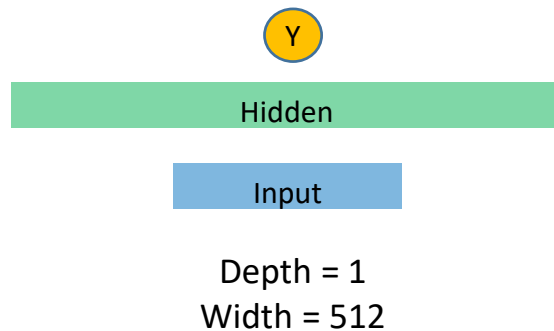
Validation splits:

- 3546 training observations
 - Split further into 80% for actual training and 20% validation
- 1000 test observations

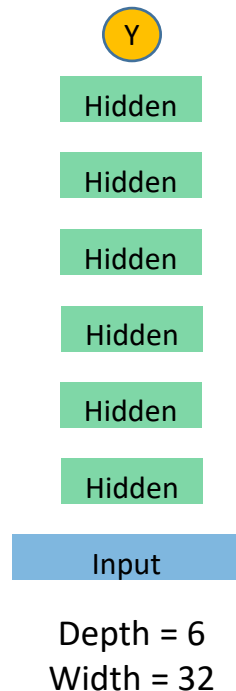
Architectures

We'll consider four architectures and a few (explicit) regularization schemes

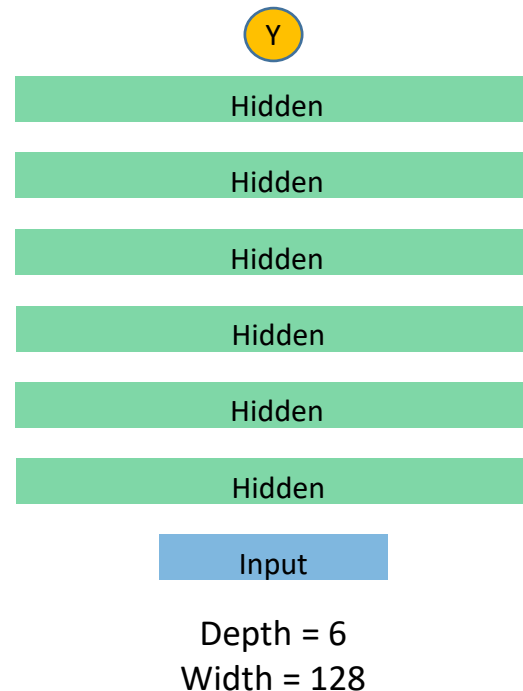
1. Wide and shallow
($p = 70145$)



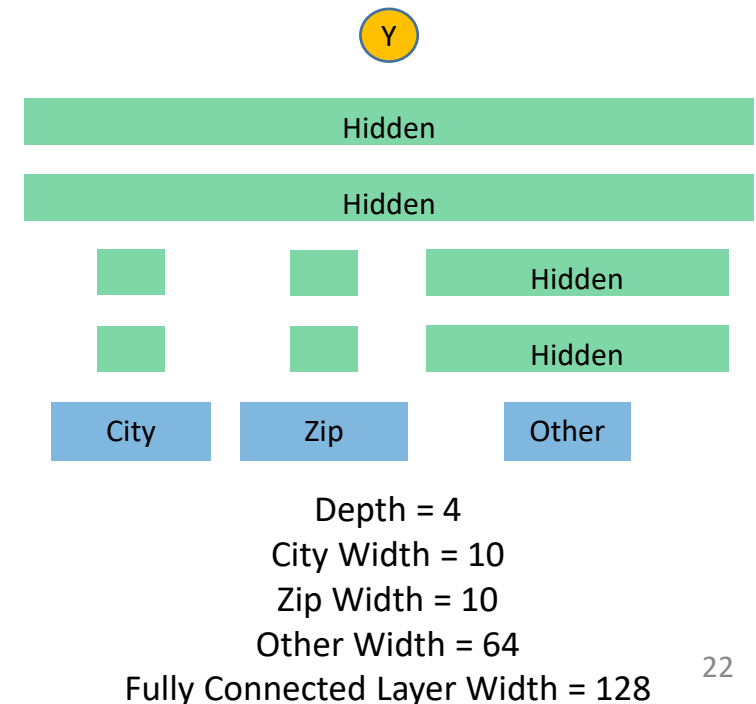
2. Deep and Narrow
($p = 9665$)



3. Deep
($p = 100097$)



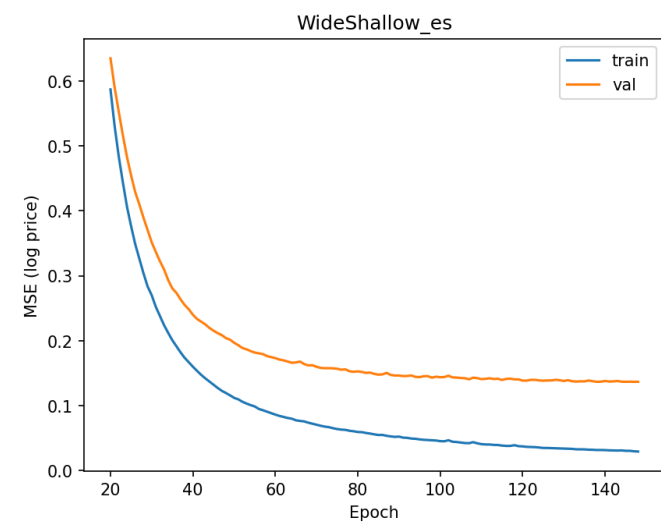
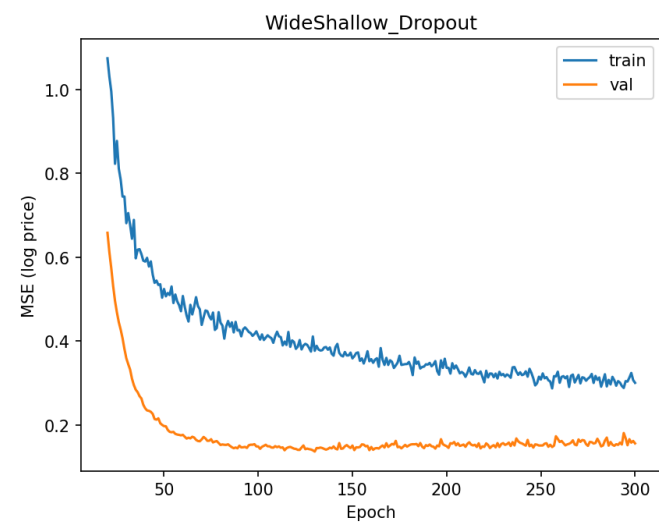
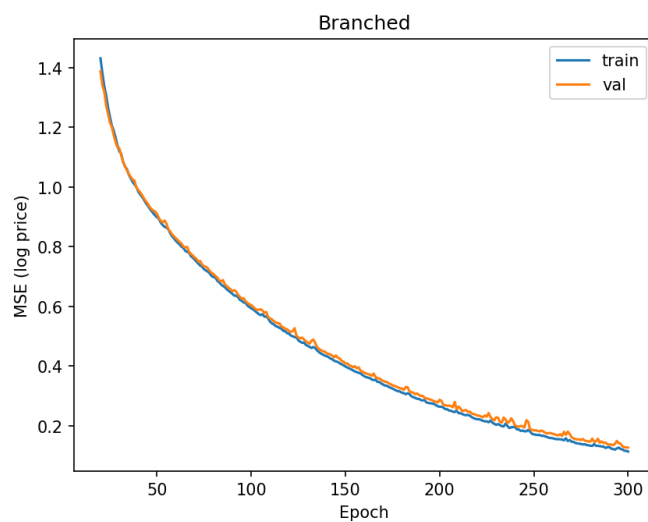
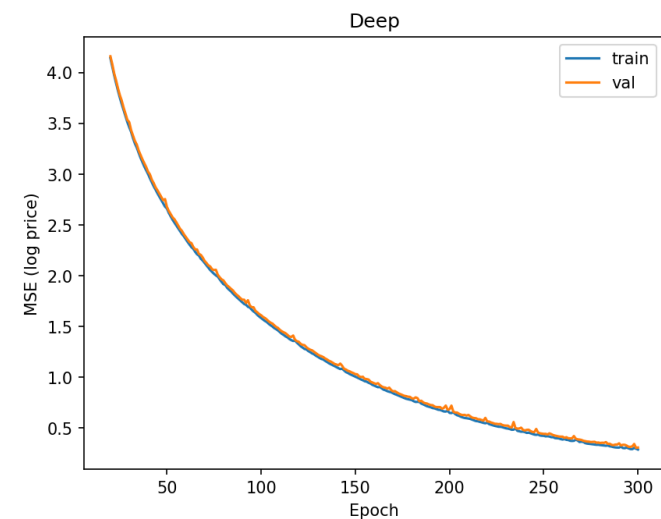
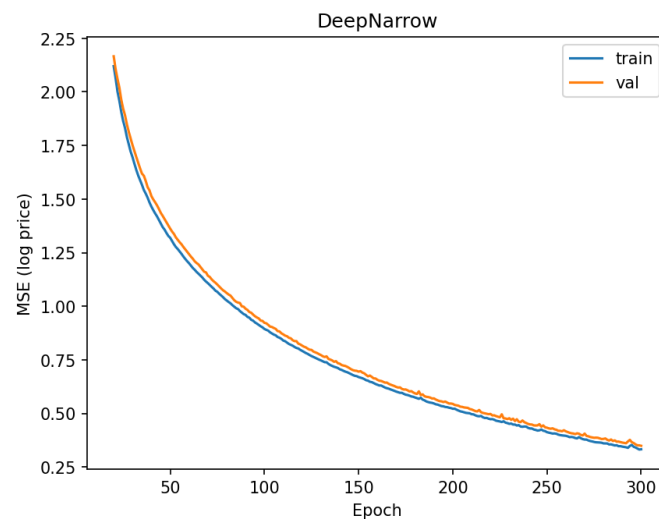
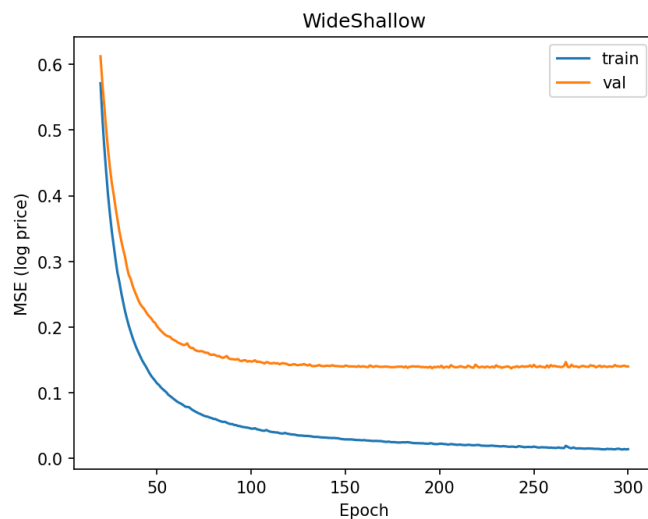
4. "Branched" (not fully connected)
($p = 34091$)



Training Choices

- Batch size: 256
- 300 epochs
 - Will try some runs with [early stopping](#)
- ℓ_2 regularization with $\lambda = .0001$
 - Only in some runs
 - If going to regularize, should think about tuning
- Dropout
 - Only in some runs
 - 30% dropout rate – should think about tuning

Training Performance



Test Performance

Model	RMSE	R ²
Linear Regression	178989	0.764
Wide Shallow	264338	0.486
Wide Shallow (Dropout)	200346	0.705
Wide Shallow (l2)	291367	0.376
Wide Shallow (Early Stopping)	255966	0.518
Deep Narrow	197905	0.712
Deep Narrow (Early Stopping)	189518	0.736
Deep	198496	0.710
Deep (Early Stopping)	191609	0.730
Branched	183307	0.753
Branched (l2)	241436	0.571
Branched (Early Stopping)	182521	0.755

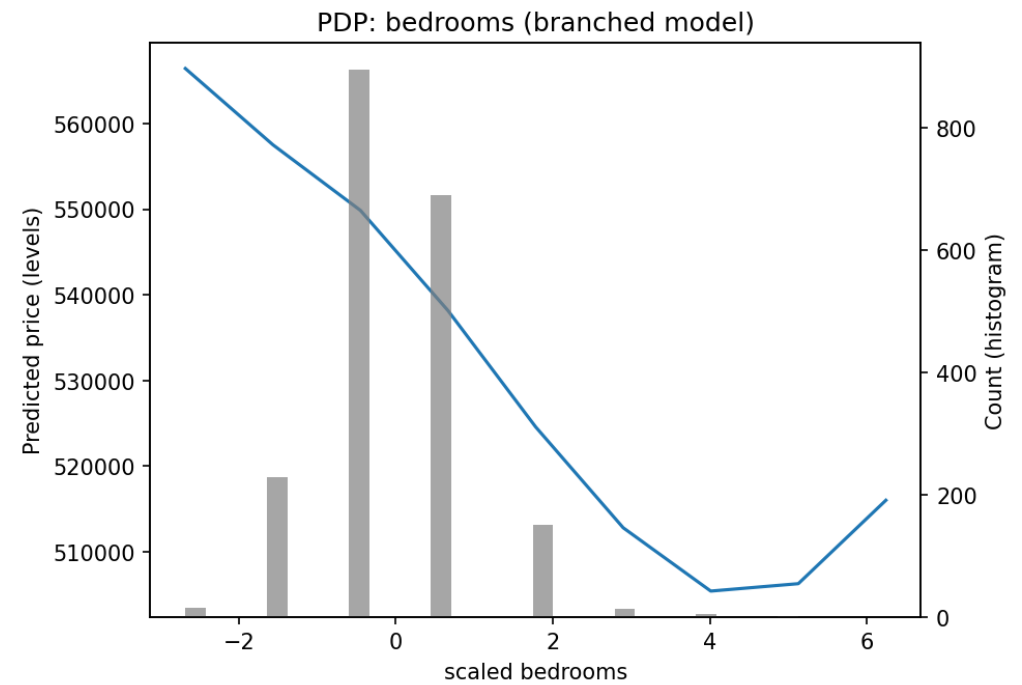
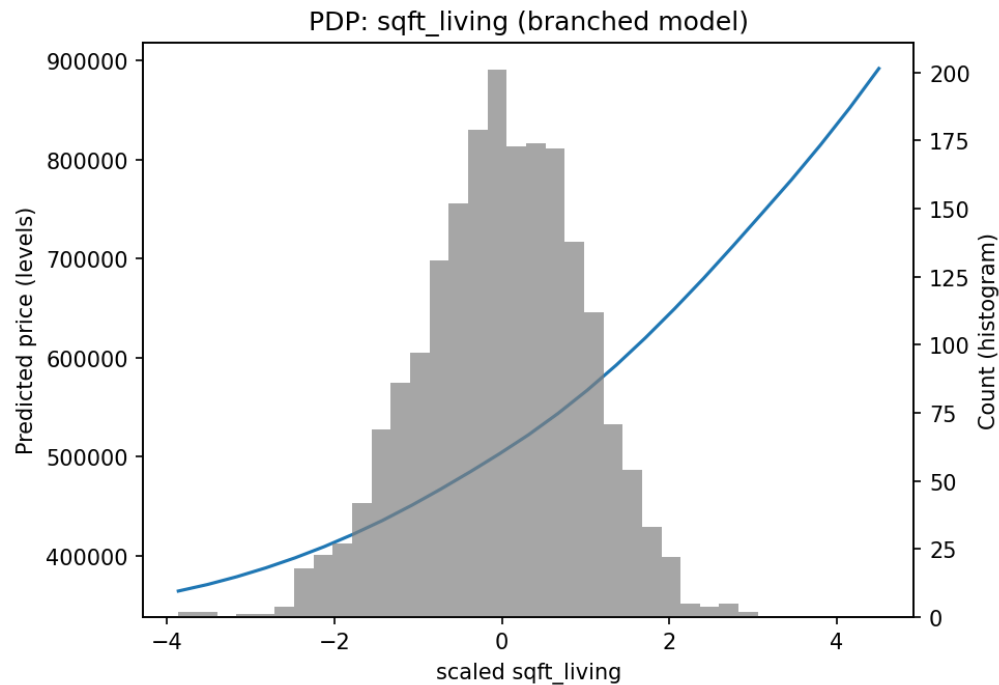
MSE and R² for price - not log(price)

Linear Regression is model from Notes 1

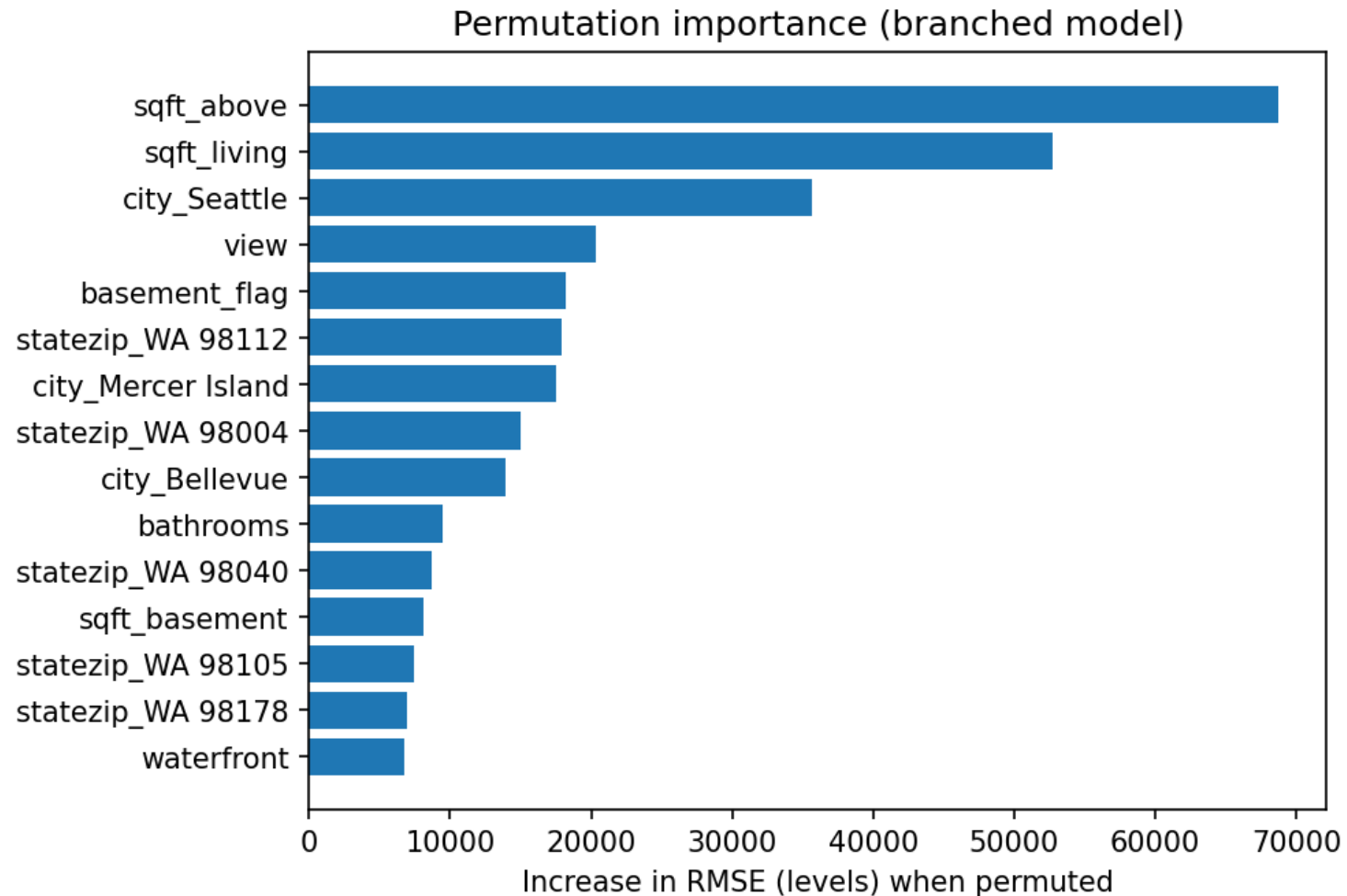
Several of the NN variants perform similarly

Partial Dependence Plots

Partial dependence plots from branched model



Variable Importance



OOS MSE $\approx$ 183K

A Bit More on Interpretation

Look at whether there are noticeable differences in predictions between groups of interest

- Might suggest model miscalibration
- Might suggest performance differences that could have downstream/disparate impacts
 - E.g. see systematically underpredicting for some group (underappraised property values)

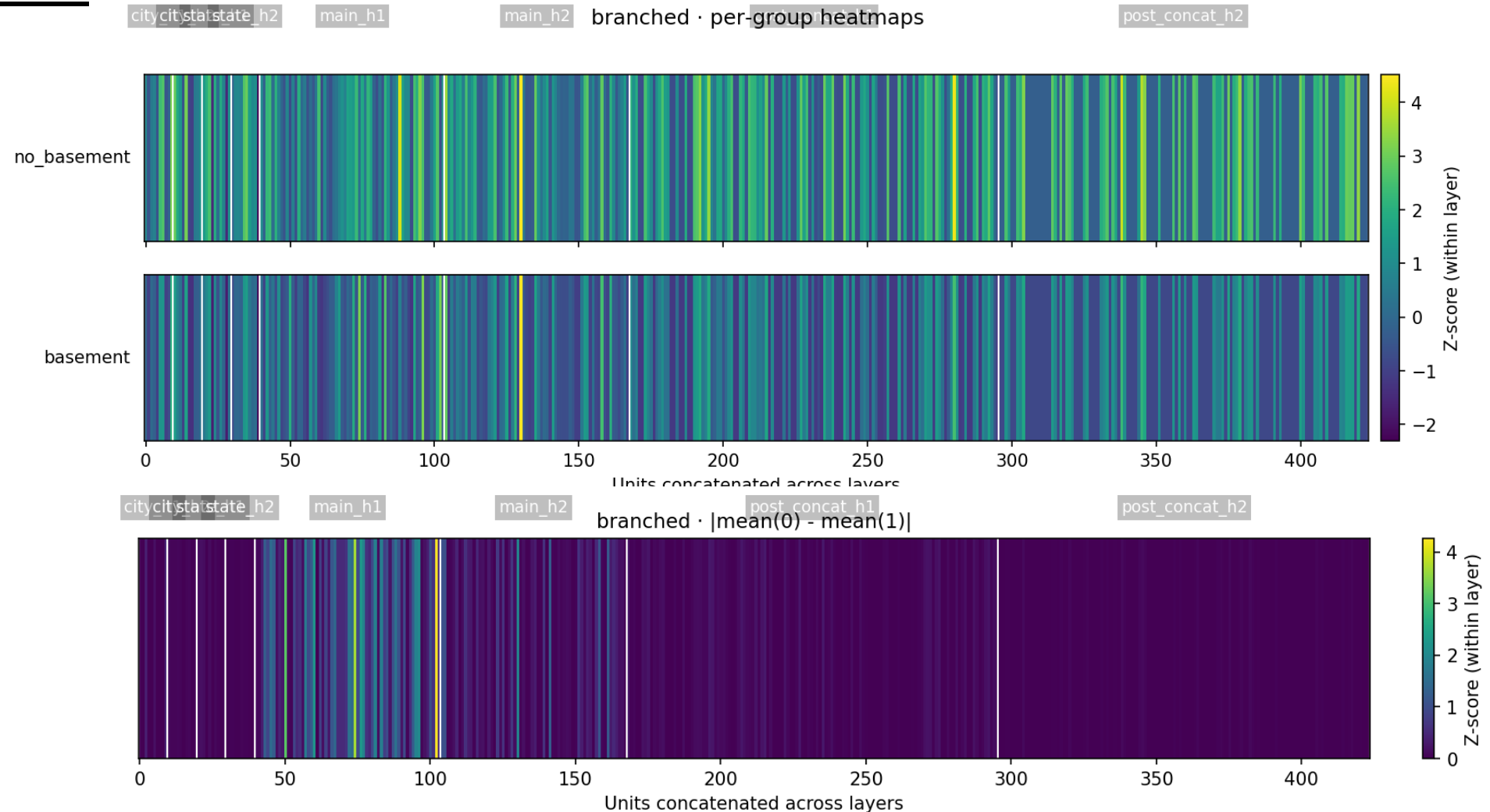
No “naturally” interesting groups here – so just going to look at presence of a basement

Basement	n	Pred. Mean	Actual Mean	Pred. Median	Actual Median	MAE	RMSE	Mean Resid
0	592	488375	492462	431677	427250	74820	125283	4087
1	408	611059	626379	525331	501662	116958	247510	15320

Activations

Can look at
neuron
activations

Here, we're
looking at “no
basement” vs
“basement”



4. Transfer Learning

Transfer Learning

DNNs powerful and useful in many cases BUT training from scratch challenging:

- Want LOTS of data for most tasks
- Want powerful, specialized computing infrastructure (GPUs, TPUs, ...)
- Lots of experimentation with architecture and other design choices

Many powerful models have already been trained

- E.g. <https://huggingface.co/>

Pre-trained models can generalize beyond their original tasks

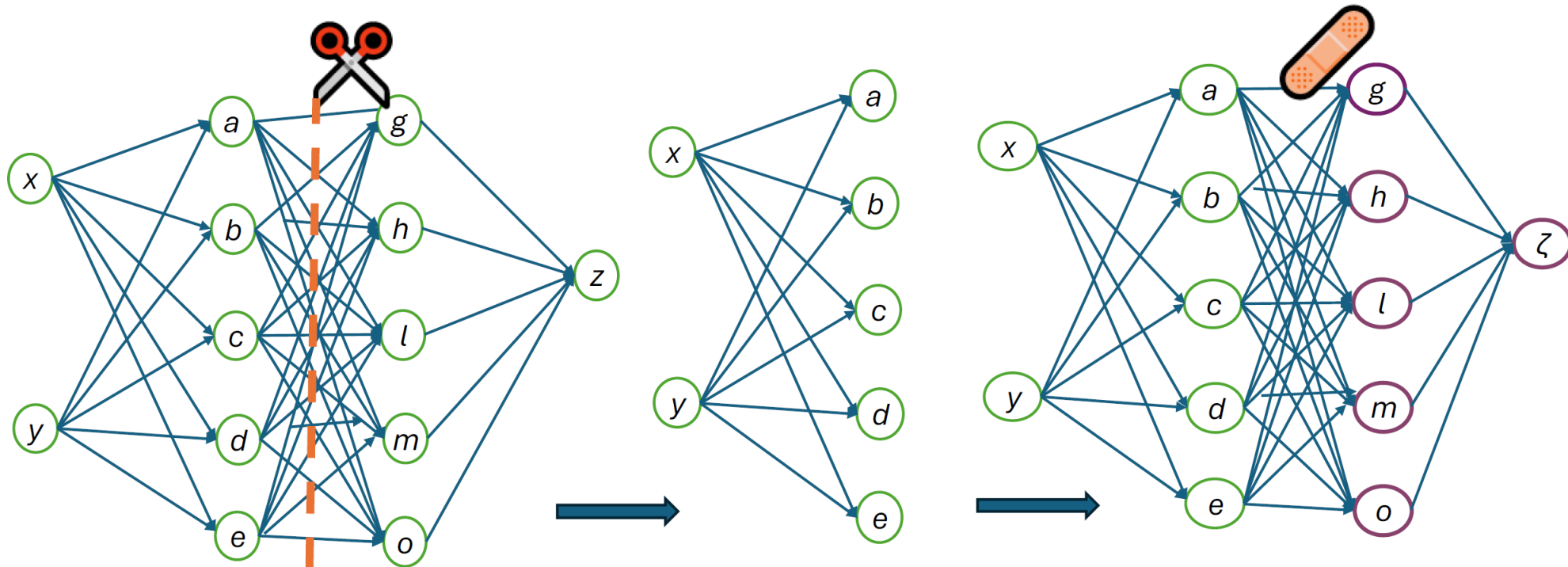
- Learned representations can be useful in related, but different, tasks

Transfer learning starts from pre-trained network and adapts it to other tasks

Transfer Learning

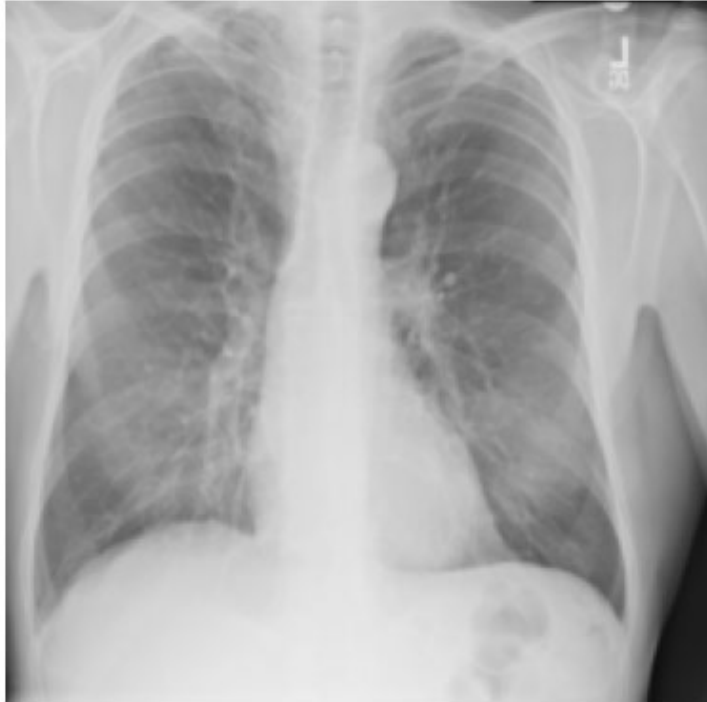
Transfer learning starts from a base network. May then

- Adapt parameters targeting a different task (**fine-tuning**)
- Reuse some of the network directly



Example: Identifying X-Ray Anomalies

Label: No Finding



Label: Infiltration



X-Ray Data

Just going to look at small subset of available data (6000 images).

- 3600 for training (fine-tuning)
- 2400 for validation

Will only look at binary classification: “No finding” vs “Finding”

Finding Labels	Follow-up #	Patient ID	Patient Age	Patient Sex	View Position
Cardiomegaly	0	1	57	M	PA
Cardiomegaly Emphysema	1	1	58	M	PA
Cardiomegaly Effusion	2	1	58	M	PA
No Finding	0	2	80	M	PA
Hernia	0	3	74	F	PA
Hernia	1	3	75	F	PA
Hernia Infiltration	2	3	76	F	PA
Hernia	3	3	77	F	PA
Hernia	4	3	78	F	PA
Hernia	5	3	79	F	PA
Hernia	6	3	80	F	PA
Hernia	7	3	81	F	PA
Mass Nodule	0	4	82	M	AP
No Finding	0	5	69	F	PA
No Finding	1	5	69	F	AP
No Finding	2	5	69	F	AP

Base Model

We're going to start from ResNet-18

- 18 layer deep residual network
 - $\approx$ 11.7 million parameters
 - Residual networks add “skip connections”
 - allow the model to bypass layers and learn adjustments instead of rebuilding from scratch
- Trained on ImageNet
 - $\approx$ 1.2 million labeled images across 1000 categories
- Developed by Microsoft Research
 - He, Zhang, Ren, and Sun (2015) “Deep Residual Learning for Image Recognition” (<https://arxiv.org/pdf/1512.03385.pdf>)
 - ResNet (not 18) winner of ImageNet 2015 competition
- Relatively lightweight
 - I can store and tune it on my laptop 😊

Do We Think It Will Work?

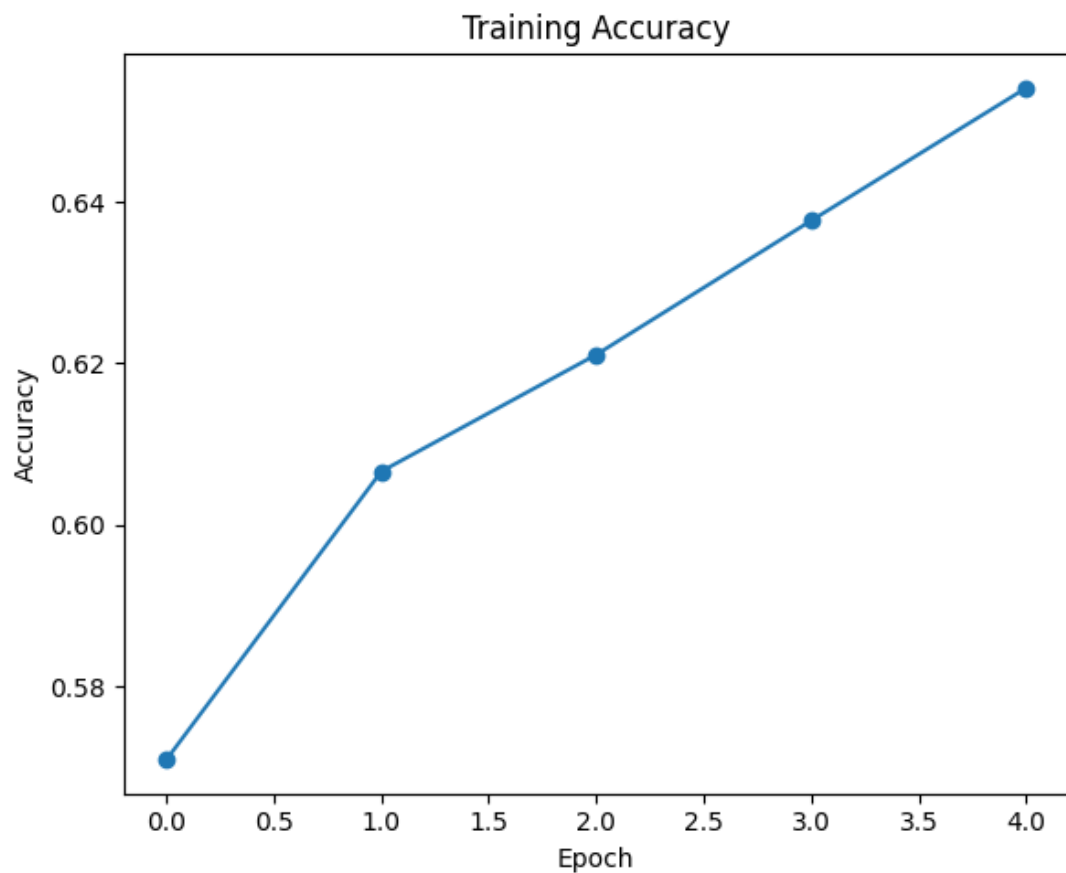
Are data and task similar to base model?

- Both datasets are of images
- ImageNet has standard objects people will immediately recognize
 - E.g., cats, dogs, ...
- X-ray data are chest X-rays
 - Interpreting X-rays often comes after years of training
 - Classifying X-rays may not be the same as classifying common images
- Maybe(?) baseline image structure learned from ImageNet useful for X-rays
 - E.g., borders, circles, connections still borders, circles, connections

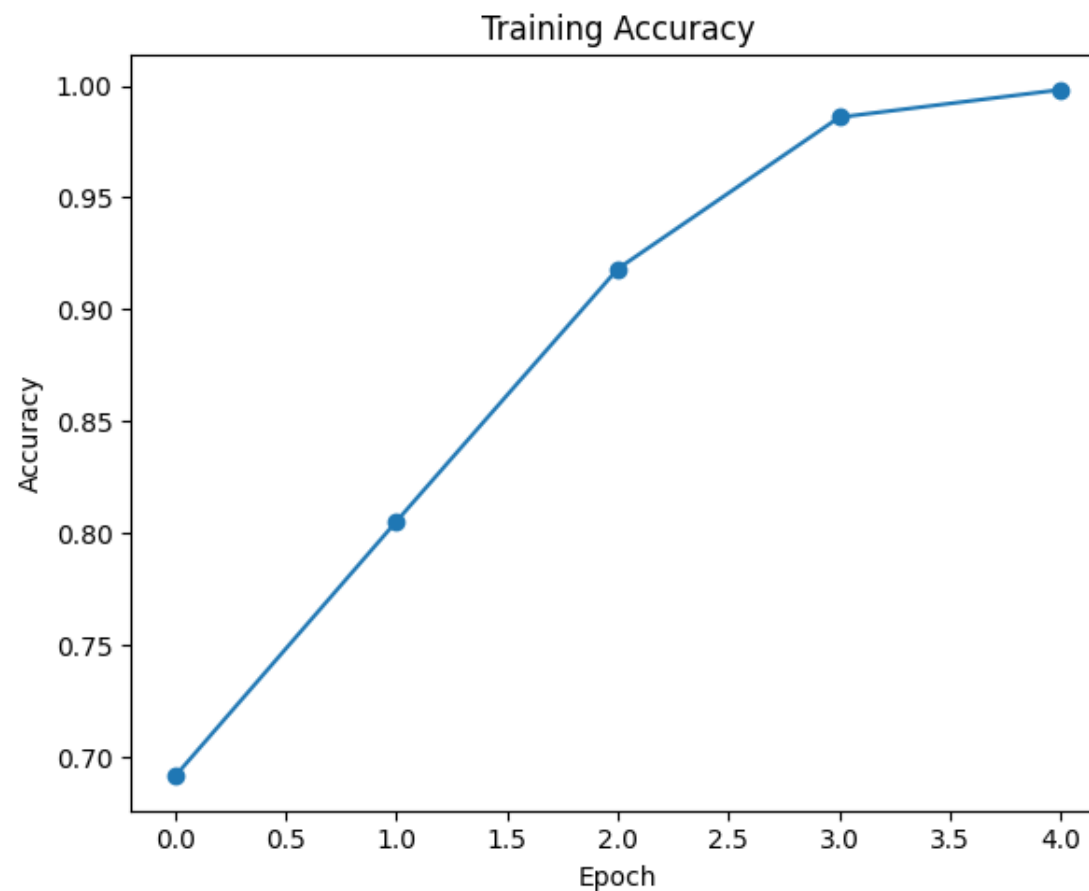
With lots of X-ray images could envision training a model from scratch

With few X-rays available, hope X-ray and ImageNet domains are “sufficiently similar”

Training



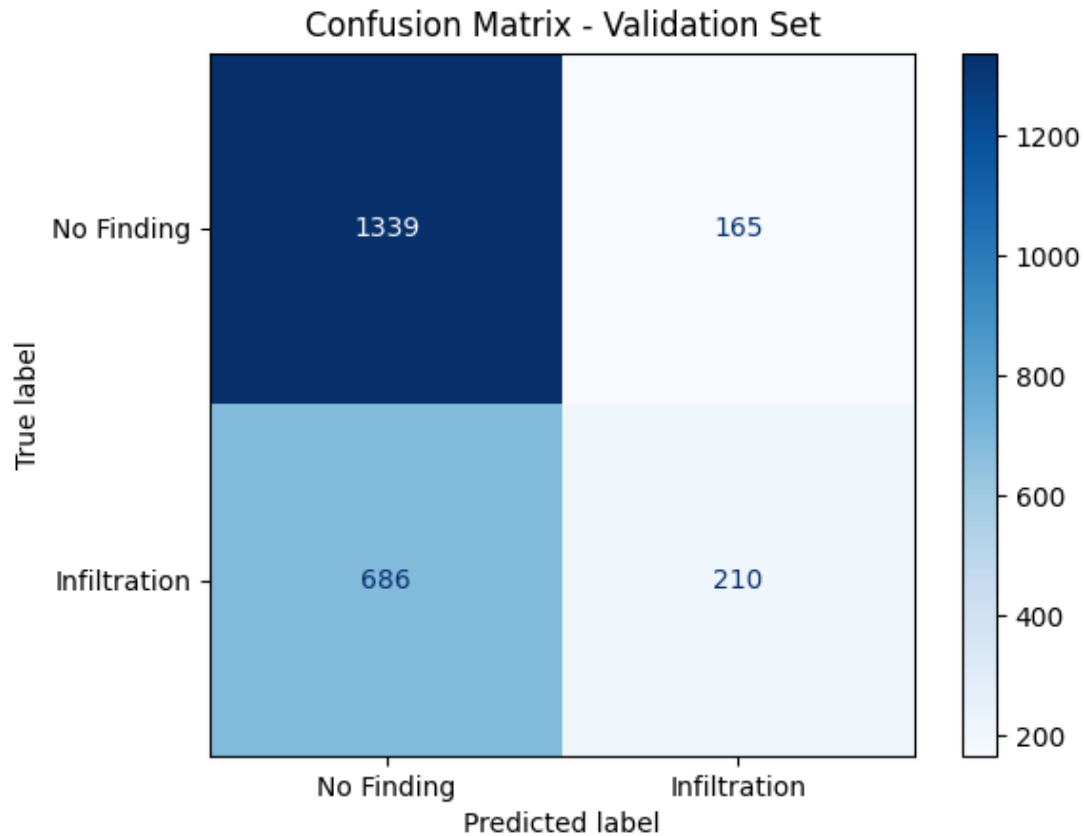
Allow only final layer parameters to update
1026 trainable parameters



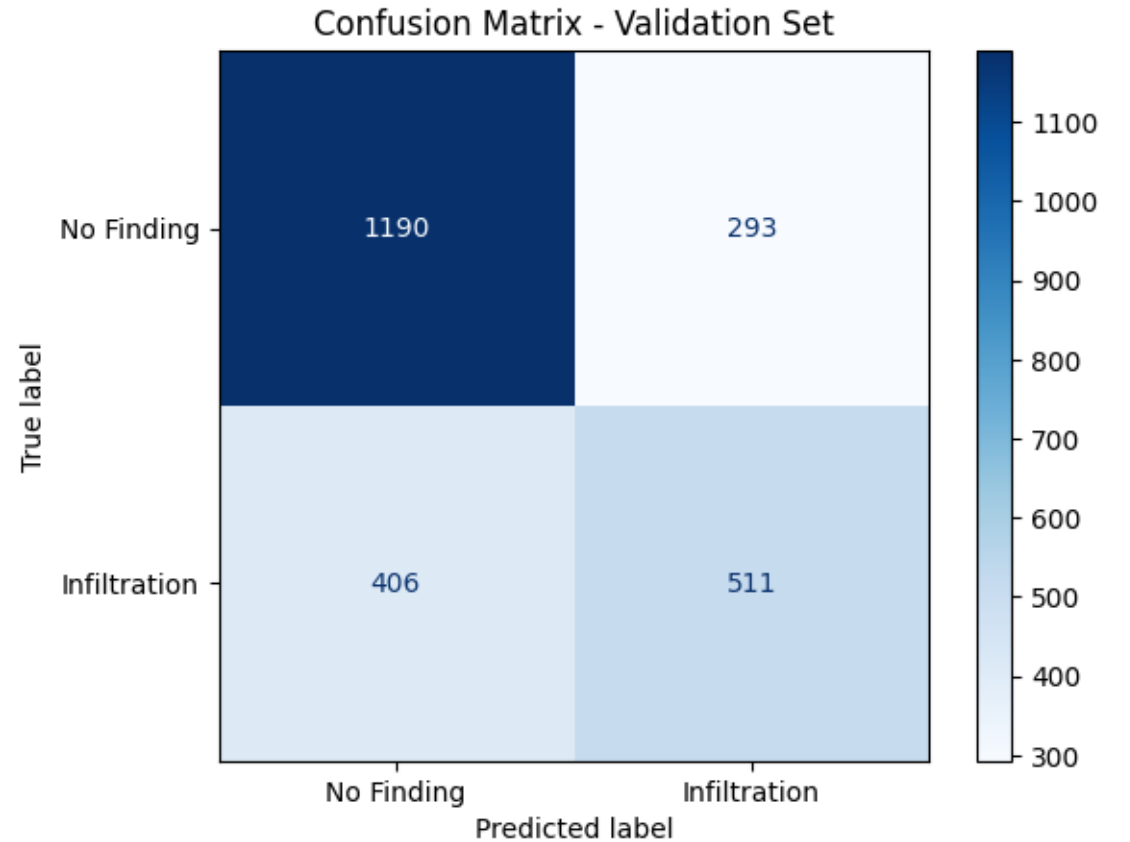
Allow all parameters to update
11,177,538 trainable parameters

Validation Performance

Note: Small differences due to RNG



Allow only final layer to update
Accuracy = 0.6454 (1549/2400)



Allow all layers to update
Accuracy = 0.7087 (1701/2400)

For reference, accuracy using just the sample mean is around 0.62. Seems like plenty of room for improvement!

Comments

ML is a major part of medical devices

- Virtually all modern imaging devices have some form of ML for classification
- Potential time/cost saver
 - E.g. many X-rays have clear results
 - ML can provide uncertainty associated with classification tasks
- FDA regulates in US with relatively stringent guidelines
- Other challenges
 - Privacy
 - Bias
 - Sufficient high-quality labeled data
 - Liability